EXTERNAL PENETRATION TESTING ASSESSMENT SIMULATION REPORT

STUDENT INFORMATION

Full Name: Tasmeoul Miyad
Student ID: 2024100000181
Serial Number: 05
Course Name: Applied Penetration Testing Engineer: Hands-on Experience.
Report For: MD. Shadman Sadique

Submission Date: 18/05/2026

"This report is submitted as part of the academic requirements for the applied penetration testing engineer course."

Executive Summary

An external penetration testing assessment was conducted against the network boundaries and web application assets of PosBuzz Retail Solutions. The core purpose of this evaluation was to perform reconnaissance, map exposed external services, identify exploitable system vulnerabilities, and conduct limited execution scenarios in authorized sandboxed environments. Testing was restricted to authorized infrastructure models provided across three TryHackMe labs: Active Reconnaissance, OWASP Juice Shop, and Blue. The results indicate a critical reliance on unpatched legacy network daemons and architectural flaws that leave web properties highly exposed to initial access campaigns.

Network Infrastructure Reconnaissance (Active Reconnaissance)

Scope and Methodology

The first phase of the assessment focused on mapping the external attack surface exposed by the target environment. The exploration relied on standard active querying network mechanisms to establish direct communication with exposed host daemons. The testing protocol followed an interactive pathway:

1. **Network Layer Verification:** Using ICMP echo loops (ping) to measure host responsiveness and confirm online execution targets.
2. **Path Enumerate Tracking:** Running traceroute procedures to define networking hops, network borders, and routing steps between the tester node and target layers.
3. **Transport Layer Mapping:** Deploying network mapping engines (nmap) to inspect open communication vectors and capture exact application headers via active banner grabbing.
4. **Interactive Port Testing:** Engaging tools like telnet and Netcat (nc) to send targeted manual requests directly to application endpoints to confirm service banners.

Key Technical Findings

1. **Exposed Communication Gateways:** Active probes verified that core web services are open externally to the public on standard communication channels (Ports 80/443).

2. **Verbose Application Blueprints:** Using browser development tools, internal client-side scripts, developer logs, and environmental details were found unmitigated inside console windows.
3. **Component Fingerprinting:** Automated extension tracking identified specific backend operational versions, database software types, and active content managers.

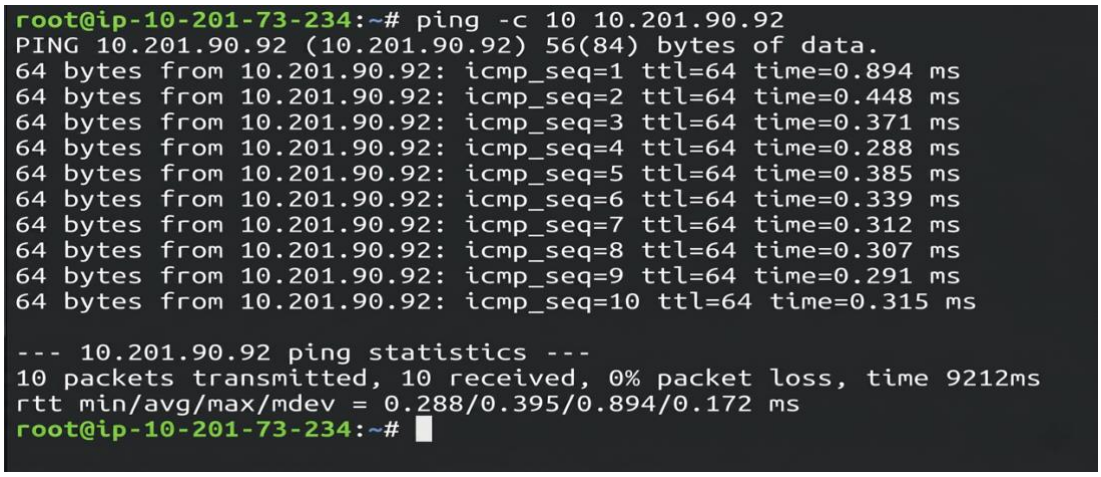
Security Impact

Exposing explicit software version strings and system architectures provides high-value intelligence to external threat actors. When granular details are leaked via connection banners or web developer consoles, malicious groups can bypass complex mapping cycles and directly correlate systems to publicly documented vulnerabilities (CVEs) and automated exploit frameworks.

Defensive Recommendations

1. **Enforce Banner Suppression:** Reconfigure web server headers and network daemons to omit version specifications (e.g., set ServerTokens ProductOnly in Apache).
2. **ICMP Network Filtering:** Set host-based and border firewalls to systematically drop external incoming ICMP Echo Requests (ping) to reduce visibility to automated port scanners.
3. **Production Code Sanitization:** Enforce strict build pipelines that automatically strip development annotations, log traces, and internal script endpoints from public code assets.

Screenshot of a POC -



Web Application Security Assessment (OWASP Juice Shop)

Scope and Methodology

The second testing scope targeted the public-facing web application layer of PosBuzz Retail Solutions. Testing was framed around verifying structural flaws defined in the OWASP Top 10 framework. The methodology implemented an audit of data sanitization loops and session state tracking:

1. **Input Interface Profiling:** Mapping public entry vectors, parameters, search controls, and authentication portals.
2. **Logic Control Testing:** Passing database-altering command strings into inputs to verify string management handling.
3. **Source Integrity Auditing:** Evaluating exposed JavaScript script structures using browser inspector units to search for configuration weaknesses.
4. **Header Variable Alteration:** Modifying implicit request headers with interception tools to verify backend log rendering engines.

Key Technical Findings

1. **Authentication Subversion (SQLi): The web platform login endpoint failed to validate system input characters. Injecting standard query statements (e.g., ' or 1=1--) forced the backend logic to true, enabling access into the principal administrator dashboard account (User ID 0) without a password.**
2. **Exposure of Restricted Dashboards: Reviewing exposed client-side configuration arrays (main-es2015.js) revealed explicit administrative panel directory pathways (/administration), allowing unauthenticated visualization of management interfaces.**
3. Header-Driven Cross-Site Scripting (XSS): The application failed to escape custom header inputs. Modifying the True-Client-IP string with frame parameters (<iframe src="javascript:alert('xss')">) forced script execution when data logs rendered inside admin screens.

Security Impact

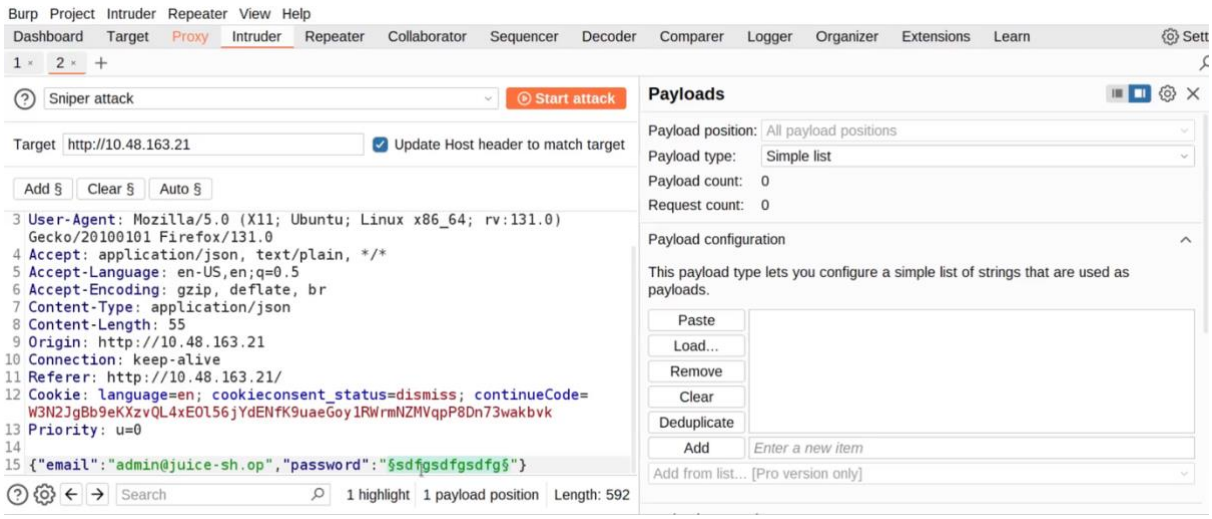
Web application boundaries represent a primary point of entry for modern threat actors. A structural failure like SQL Injection allows malicious users to fully subvert authentication models, bypass local permissions, alter system configurations, or extract confidential user databases. Furthermore, weak

directory tracking and client-side code exposure reveal administrative pages that can be used to execute cross-site script payloads to compromise administrative sessions.

Defensive Recommendations

1. **Mandate Parameterized Statements: Transition backend coding models away from dynamic query concatenation. Enforce prepared statements universally across all user-supplied endpoints.**
2. Server-Side Access Enforcement: Remove administrative pathway strings from public client-side JavaScript assets. Enforce role-based access checking on the server architecture for all privileged endpoints.
3. **Context-Aware Encoding: Sanitize and escape all user inputs, header strings, and database logs contextually before rendering data onto any UI viewport.**

Screenshot of a POC –



OS Vulnerability Validation & Exploitation (Blue)

Scope and Methodology

The final testing scope involved verifying underlying infrastructure configurations on the target platform. The methodology focused on analyzing unpatched operating system layers and validation of high-severity legacy vulnerabilities:

1. **Target Port Identification:** Running targeted script scans across standard file-sharing architectures (Port 445 SMB) to evaluate vulnerability profiles.
2. **Controlled Exploit Deployment:** Using specialized testing platforms Metasploit to inject test data payloads into identified system buffers.
3. **Session Privilege Upgrading:** Deploying operational commands inside active terminals to establish system-level access paths.
4. **Post-Exploitation Configuration Auditing:** Accessing systemic data fields and credential stores to determine total local compromise.

Key Technical Findings

1. **Critical Legacy Protocol Exposure (MS17-010):** Network enumeration verified that the system was running an unpatched version of the Server Message Block v1 (SMBv1) protocol, making it explicitly vulnerable to the **EternalBlue** vulnerability family.
2. **System Foothold Acquisition:** Exploiting the kernel memory structure using the Metasploit module exploit/windows/smb/ms17_010_eternalblue successfully yielded an unauthenticated reverse command terminal.
3. **Maximum Privilege Escalation:** Upgrading the terminal structure and deploying the getsystem tool successfully elevated permissions to local kernel control: **NT AUTHORITY\SYSTEM**.
4. **Credential Credential Extraction:** Accessing the privileged memory environment allowed process migration and execution of a hashdump query, completely exposing the local Security Account Manager (SAM)

database password hashes.

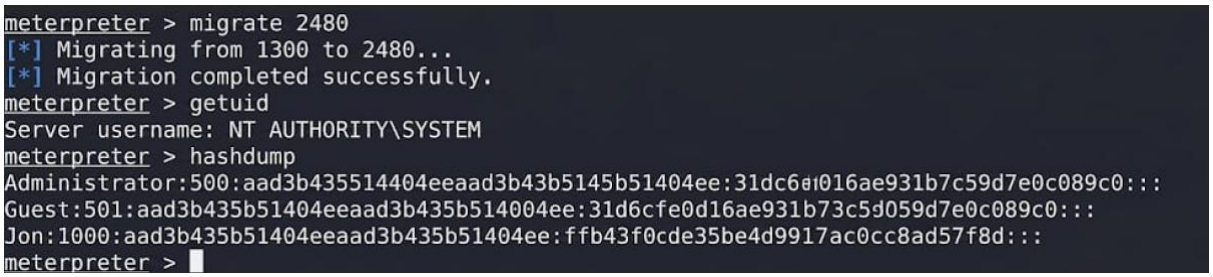
Security Impact

The presence of unpatched MS17-010 vulnerabilities represents a severe security failure. Because this exploit targets kernel memory management flaws via unauthenticated network packets, an external actor can execute random code strings with maximum machine privileges. This allows an attacker to manipulate core operating files, extract password stores, deploy rootkits, or move laterally across the internal enterprise network as a self-propagating worm.

Defensive Recommendations

1. **Disable SMBv1 Native Architecture:** Issue network-wide Group Policy Objects (GPO) to completely disable SMBv1 across all system endpoints. Mandate the use of modern SMBv2 or SMBv3 alternatives.
2. **Accelerated Security Patch Management:** Deploy the Microsoft patch bulletin MS17-010 instantly to all vulnerable endpoints and maintain a rigid monthly patch verification schedule.
3. **Network Firelocking Enforcements:** Restrict inbound connection rules on perimeter firewalls to drop Port 445 (SMB) queries originating from external or public network interfaces.

Screenshot of a POC -



End of Report.